

%

$\neq$

Bool

Exclusive Divisibility Logic

Solving the “Exactly One” Problem in Python

∇

∇

$\oplus$

FUNCTION SPECIFICATION

```
def div_by_exactly_one(num: int, a: int, b: int) -> bool:
```

INPUTS

num : The integer to evaluate

a : First divisor

b : Second divisor

OUTPUT LOGIC

Return True if 'num' is divisible by exactly one of 'a' or 'b'.

Return False if divisible by both.

Return False if divisible by neither.

Constraint: Pure function logic only. No I/O operations required.

Scenario Testing: The Logic Trap

CASE 1: Success

num = 20, a = 5, b = 6

Divisible by 5? YES ✓

Divisible by 6? NO ✗

Result: True

CASE 2: The Trap

num = 20, a = 5, b = 10

Divisible by 5? YES ✓

Divisible by 10? YES ✓

Result: False ← Exclusive
means
ONE only.
Both = Fail.

CASE 3: Failure

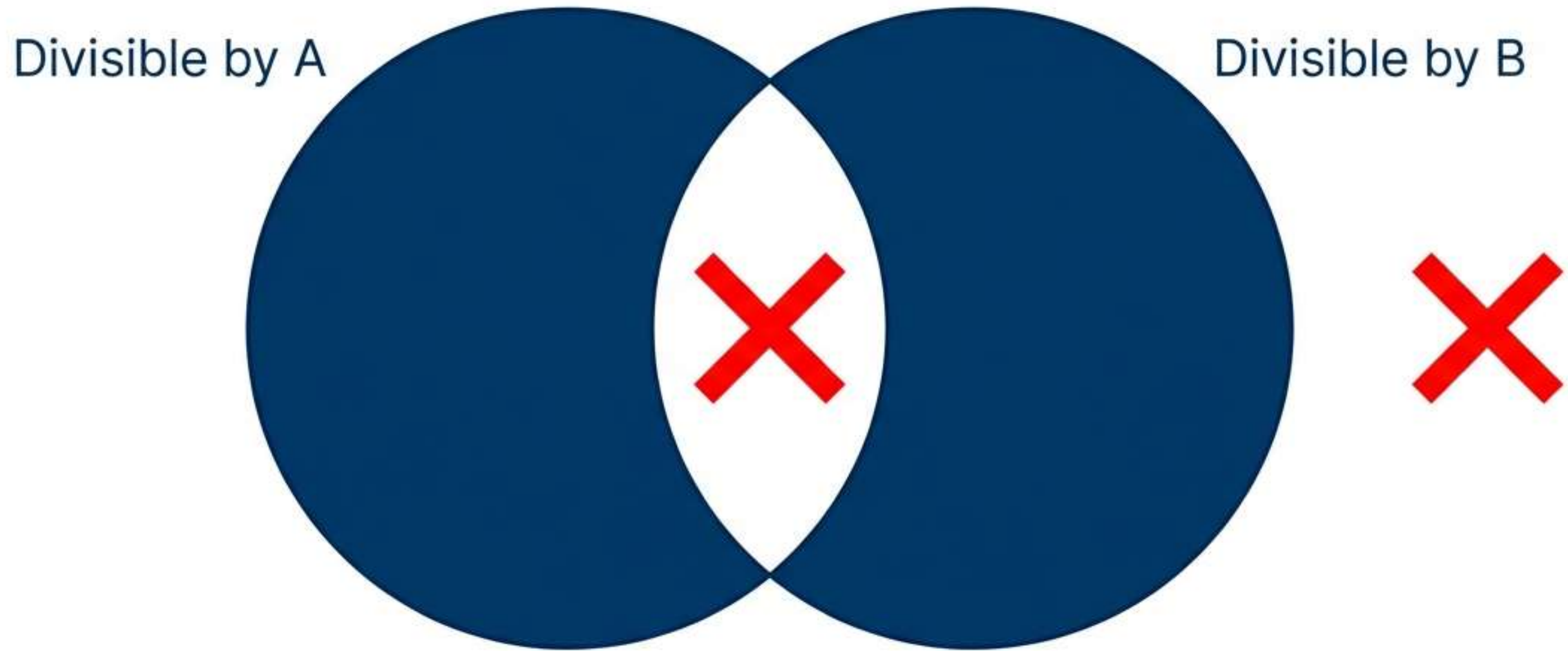
num = 20, a = 6, b = 7

Divisible by 6? NO ✗

Divisible by 7? NO ✗

Result: False

Decoding “Exactly One” (XOR)



Logic: $(A \text{ AND NOT } B) \text{ OR } (B \text{ AND NOT } A)$

Commonly known as: Exclusive OR

The Tool: Modulo Operator

%

Returns the Remainder

Check:

`num % n == 0`

Remainder is zero
means perfectly divisible.

`20 % 5 == 0 → True`

`20 % 6 != 0 → True (Remainder is 2)`

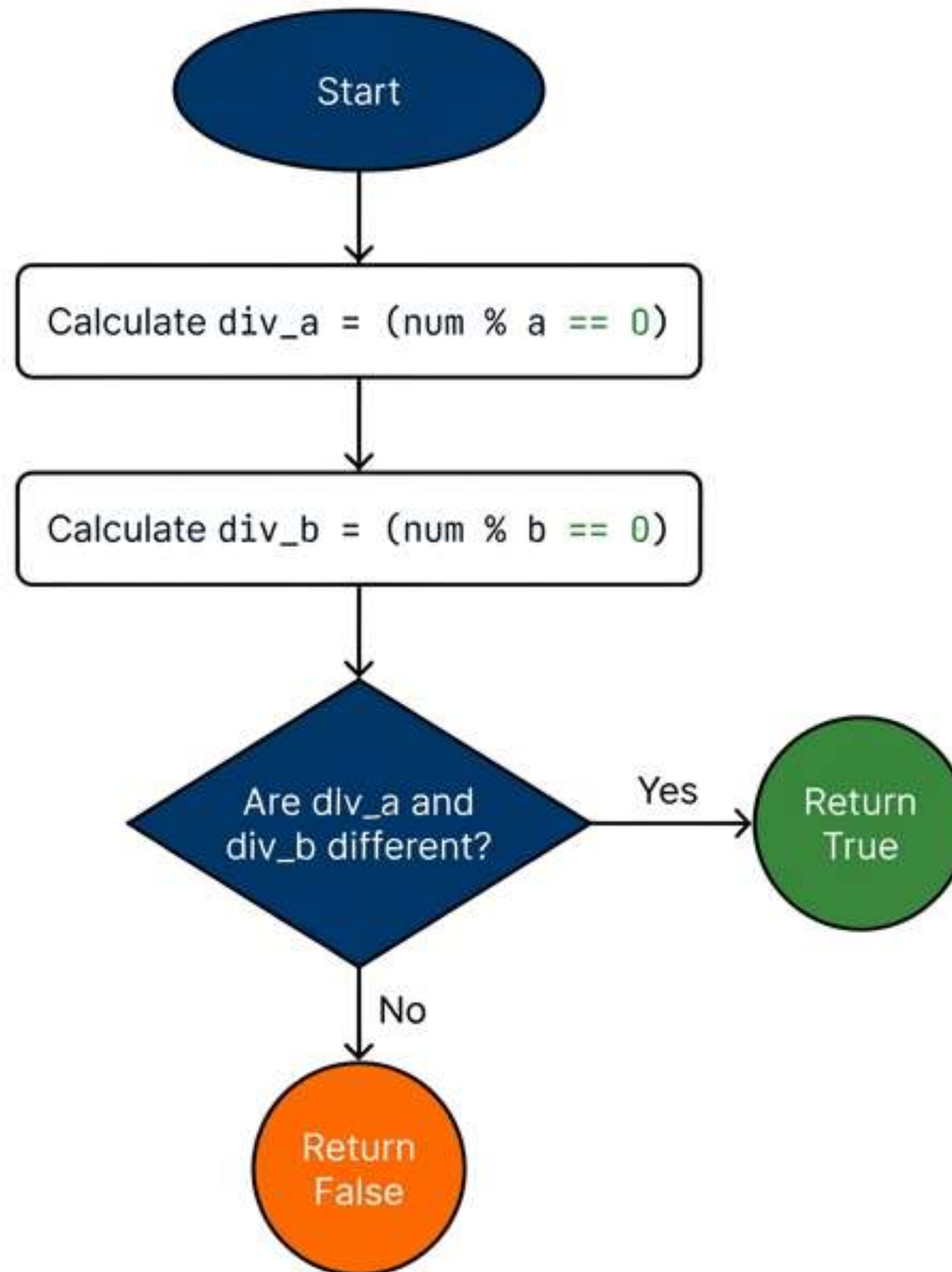
The Logic Proof: Truth Table

Divisible by A	Divisible by B	Result
True	False	True (Valid)
False	True	True (Valid)
True	True	False (Invalid)
False	False	False (Invalid)



Notice the pattern?
The result is True
only when the inputs
are DIFFERENT.

The Algorithm Flowchart



Approach 1: The Explicit Logic

Translating the requirements directly into English-like code.

```
def div_by_exactly_one(num: int, a: int, b: int) -> bool:
    # Check explicit conditions
    if (num % a == 0 and num % b != 0) or \ ← Matches: A is True AND B is False
        (num % b == 0 and num % a != 0): ← Matches: B is True AND A is False
        return True
    return False
```

Status: Correct, but Verbose.

The Insight: Boolean Equality

True	!=	False	→	True
False	!=	True	→	True
True	!=	True	→	False
False	!=	False	→	False



Therefore:
**Exactly One ==
Not Equal (!=)**

We don't need to check each case manually. We just need to check if the boolean results are different.

Approach 2: The Pythonic Solution

Applying the Boolean Equality insight.

```
def div_by_exactly_one(num: int, a: int, b: int) -> bool:  
    div_a = (num % a == 0)  
    div_b = (num % b == 0)  
    return div_a != div_b
```

→ When acting on Booleans,
"!=" functions as XOR.

Code Comparison

Explicit Approach

```
if (div_a and not div_b) or...
```

- Readable but Verbose.
- Maintains logical steps.

Pythonic Approach

```
return div_a != div_b
```

- Concise & Elegant.
- Uses Boolean Algebra.

Time Complexity for both: $O(1)$

Key Takeaways



Modulo (%): checks divisibility by looking for a **remainder of 0**.



Logic Trap: 'Exactly One' is **NOT the same as** 'One or Both'. It is strict XOR.



Boolean Math: The Not Equal operator ($\neq$) acts as a shortcut for XOR when comparing True/False values.



Efficiency: Eliminating branching (if/else) creates **cleaner, more professional code**.

Final Reference Code

```
def div_by_exactly_one(num: int, a: int, b: int) -> bool:
    '''Check if num is divisible by exactly one of a or b.
    Args:
        num, a, b : int - input numbers
    Returns:
        bool - True if num is divisible by exactly one of a or b.
    '''
    return (num % a == 0) != (num % b == 0)
```

End of Module.